
Cuadernillo de Estudio — Clase 8

Identificación de Vulnerabilidades: De la Teoría al Caso Real

Seguridad Aplicada a Sistemas de Información
Licenciatura en Sistemas de Información — Facultad de Informática y Diseño
Docente: Ing. Rodrigo Atilio Elgueta
Ciclo Académico Vigente

Índice

1. Objetivos de aprendizaje	3
2. Vulnerabilidades: Definición y Taxonomía	3
2.1. Definición formal	3
2.2. Estándares de clasificación	4
2.3. Escala de severidad CVSS	4
3. Vulnerabilidades en Componentes de Infraestructura	4
3.1. Arquitectura típica de una aplicación web	4
3.2. Balanceador de Carga	5
3.3. Terminador de Túneles SSL / Proxy Inverso	5
3.4. WAF (Web Application Firewall)	5
3.5. Web Server / App Server	5
3.6. Base de Datos	5
4. Causas Raíz de las Vulnerabilidades	6
5. Metodología de Identificación de Vulnerabilidades	6
6. Caso de Estudio para la Actividad A07	7
7. Glosario de conceptos clave	8
8. Autoevaluación	9
9. Respuestas de la Autoevaluación	9
10. Actividad A07: Identificación de Vulnerabilidades	11

11.Fuentes y bibliografía	12
11.1. Bibliografía obligatoria (según programa)	12
11.2. Bibliografía complementaria (según programa)	12
11.3. Bibliografía específica de vulnerabilidades	13
11.4. Recursos web y laboratorios	13

1. Objetivos de aprendizaje

Al finalizar la lectura y el trabajo con este cuadernillo, el estudiante será capaz de:

- Definir formalmente el concepto de **vulnerabilidad** y diferenciarlo de amenaza, riesgo y exploit.
- Clasificar vulnerabilidades según estándares internacionales (**CVE**, **CVSS**, **CWE**).
- Identificar vulnerabilidades típicas en **componentes de infraestructura** (balanceadores, WAF, servidores web, bases de datos).
- Analizar las **causas raíz** que generan vulnerabilidades en software, hardware, redes y comunicaciones.
- Aplicar el conocimiento a un **caso real** integrando herramientas de diagnóstico vistas en la Actividad A06.
- Preparar la **Actividad A07** (Informe de identificación de vulnerabilidades con revisión de caso).

Cómo usar este cuadernillo

Esta clase es la bisagra entre la teoría de herramientas (Clase 7) y la práctica de explotación (Clases 9-10). Aquí aprenderás a **identificar y documentar** vulnerabilidades antes de explotarlas. Las respuestas de la autoevaluación están en la Sección 9.

2. Vulnerabilidades: Definición y Taxonomía

2.1. Definición formal

Vulnerabilidad

Debilidad o falla en el diseño, implementación, operación o gestión de un sistema que puede ser **explotada** por una amenaza para comprometer los activos de información.

Exploit

Código, herramienta o técnica que **aprovecha** una vulnerabilidad específica para lograr un resultado no deseado (acceso no autorizado, ejecución de código, denegación de servicio).

Riesgo de una vulnerabilidad

Combinación de la **probabilidad** de que la vulnerabilidad sea explotada y el **impacto** que dicha explotación tendría sobre los activos de la organización.

Relación entre conceptos

- **Vulnerabilidad** → La debilidad del sistema (el «agujero»).
- **Exploit** → La herramienta que aprovecha esa debilidad (la «llave»).
- **Riesgo** → La probabilidad de que alguien use esa llave y el daño que cause.
- **Salvaguarda** → La medida que cierra o reduce el agujero.

2.2. Estándares de clasificación

CVE — Common Vulnerabilities and Exposures

Catálogo público de vulnerabilidades conocidas mantenido por MITRE. Cada vulnerabilidad recibe un identificador único en formato **CVE-AÑO-NÚMERO** (ej. CVE-2021-44228 para Log4Shell).

CVSS — Common Vulnerability Scoring System

Sistema de puntuación estándar (0 a 10) que mide la **severidad** de una vulnerabilidad considerando:

- **Vector de ataque:** ¿Es remota o requiere acceso local?
- **Complejidad:** ¿Qué tan difícil es explotarla?
- **Autenticación:** ¿Requiere credenciales?
- **Impacto:** ¿Afecta confidencialidad, integridad y/o disponibilidad?

CWE — Common Weakness Enumeration

Clasificación de **tipos de debilidades** de software (causas raíz). Por ejemplo:

- CWE-89: Inyección SQL
- CWE-79: Cross-Site Scripting (XSS)
- CWE-287: Autenticación inadecuada
- CWE-798: Uso de credenciales hardcodeadas

2.3. Escala de severidad CVSS

Severidad	CVSS	Acción recomendada
Crítica	9.0 – 10.0	Corrección inmediata. Puede indicar explotación activa.
Alta	7.0 – 8.9	Corrección prioritaria en el próximo ciclo de parches.
Media	4.0 – 6.9	Planificar corrección según riesgo residual.
Baja	0.1 – 3.9	Monitorear y evaluar si aplica al contexto.

3. Vulnerabilidades en Componentes de Infraestructura

En la Unidad III vimos la seguridad lógica por capas del modelo OSI. Ahora profundizamos en los **componentes habituales** de una infraestructura web corporativa y sus vulnerabilidades típicas.

3.1. Arquitectura típica de una aplicación web

Usuario → Balanceador → WAF → Web Server → App Server → Base de Datos

3.2. Balanceador de Carga

- **Función:** Distribuir el tráfico entre múltiples servidores para garantizar disponibilidad.
- **Vulnerabilidades típicas:**
 - Configuración de session persistence débil (permite session hijacking).
 - Exposición de puertos de administración sin cifrado.
 - Falta de límites de tasa (rate limiting) que permite ataques DDoS.

3.3. Terminador de Túneles SSL / Proxy Inverso

- **Función:** Manejar el cifrado TLS, actuar como punto único de entrada.
- **Vulnerabilidades típicas:**
 - Certificados SSL/TLS desactualizados o con protocolos inseguros (SSLv3, TLS 1.0).
 - Configuraciones que permiten ataques de downgrade (POODLE, BEAST).
 - Heartbleed (CVE-2014-0160): vulnerabilidad en OpenSSL que permitía leer memoria del servidor.

3.4. WAF (Web Application Firewall)

- **Función:** Filtrar tráfico HTTP/HTTPS malicioso antes de llegar al servidor.
- **Vulnerabilidades típicas:**
 - Reglas mal configuradas (bypass de firmas).
 - Evasión mediante encoding múltiple o HTTP Parameter Pollution.
 - Falta de actualización de reglas para nuevas amenazas.

3.5. Web Server / App Server

- **Función:** Servir contenido estático (HTML, CSS) y ejecutar lógica de aplicación.
- **Vulnerabilidades típicas:**
 - Software desactualizado (Apache, Nginx, Tomcat, IIS).
 - Directorios expuestos sin autenticación.
 - Archivos de configuración con credenciales visibles.
 - Inyección de comandos del sistema operativo (OS Command Injection).

3.6. Base de Datos

- **Función:** Almacenar y gestionar los datos de la aplicación.
- **Vulnerabilidades típicas:**
 - Inyección SQL (CWE-89): manipulación de consultas.
 - Credenciales por defecto o hardcodeadas (CWE-798).
 - Puertos expuestos innecesariamente a Internet.
 - Falta de cifrado de datos en reposo.

OWASP Top 10

El proyecto OWASP publica periódicamente las **10 vulnerabilidades más críticas** en aplicaciones web. Las más recurrentes incluyen:

1. Broken Access Control (Control de acceso roto)
2. Cryptographic Failures (Fallas criptográficas)
3. Injection (Inyección SQL, de comandos, etc.)
4. Insecure Design (Diseño inseguro)
5. Security Misconfiguration (Configuración errónea de seguridad)
6. Vulnerable and Outdated Components (Componentes desactualizados)
7. Identification and Authentication Failures (Fallas de identificación y autenticación)
8. Software and Data Integrity Failures (Fallas de integridad)
9. Security Logging and Monitoring Failures (Fallas en logs y monitoreo)
10. Server-Side Request Forgery (SSRF)

4. Causas Raíz de las Vulnerabilidades

Las vulnerabilidades no aparecen por magia. Tienen causas identificables:

Categoría	Causa típica	Ejemplo
Diseño	Falta de análisis de seguridad en la arquitectura	Sistema diseñado sin segmentación de red
Desarrollo	Falta de validación de entradas, manejo inseguro de memoria	Buffer overflow, inyección SQL
Configuración	Parámetros por defecto, servicios innecesarios habilitados	Puertos abiertos, usuarios default
Operación	Falta de parches, actualizaciones no aplicadas	Vulnerabilidades conocidas sin parchar
Humanos	Errores, ingeniería social, falta de capacitación	Caer en phishing, compartir credenciales

5. Metodología de Identificación de Vulnerabilidades

Un pentester profesional sigue un proceso estructurado para identificar vulnerabilidades:

1. **Recopilación de información:** Identificar activos, tecnologías, versiones de software.
2. **Escaneo automatizado:** Usar herramientas como Nmap, Nikto, OpenVAS para detectar vulnerabilidades conocidas.
3. **Análisis manual:** Revisar configuraciones, código fuente, lógica de negocio.

4. **Verificación:** Confirmar que la vulnerabilidad es explotable (sin causar daño).
5. **Clasificación:** Asignar CVE, puntuación CVSS y prioridad.
6. **Documentación:** Redactar el informe con evidencia y recomendaciones.

Integración con A06: Herramientas de Kali para identificación

En la Actividad A06 investigaste herramientas. Aquí te mostramos cómo se integran en el flujo de identificación:

- **Nmap:** Escaneo de puertos y detección de servicios/versiones.

```
1 nmap -sV -sC --script=vuln 192.168.1.0/24
```

- **Nikto:** Escaneo de vulnerabilidades web conocidas.

```
1 nikto -h http://target.com -Tuning 1234567890
```

- **OpenVAS (GVM):** Escáner de vulnerabilidades completo con reportes CVSS.

- **Searchsploit:** Búsqueda offline de exploits para vulnerabilidades encontradas.

```
1 searchsploit apache 2.4.49
```

6. Caso de Estudio para la Actividad A07

Caso: «El Portal de la Municipalidad»

La Municipalidad de *Valle Grande* tiene un portal web institucional donde los ciudadanos consultan el estado de sus trámites. El portal fue desarrollado por una consultora externa y no se le realizan auditorías de seguridad desde su implementación.

Situación observada durante un análisis preliminar:

- El servidor web corre **Apache 2.4.29** (desactualizado).
- Existe un directorio `/admin` accesible sin autenticación.
- El formulario de login envía credenciales por **HTTP plano** (sin HTTPS).
- La base de datos MySQL está expuesta en el puerto 3306 hacia Internet.
- El certificado SSL del sitio está vencido.
- Se encontró un archivo `config.php.bak` con credenciales de BD visibles.
- El servidor tiene habilitado el módulo `mod_status` de Apache accesible públicamente.

Consigna de A07: Elaborar un informe profesional que:

1. Identifique y clasifique cada vulnerabilidad (tipo CWE, severidad CVSS estimada).
2. Integre al menos una herramienta investigada en A06 para demostrar cómo se detectaría cada hallazgo.
3. Proponga salvaguardas técnicas y administrativas.
4. Evalúe el riesgo residual tras la aplicación de las salvaguardas.

7. Glosario de conceptos clave

Vulnerabilidad	Debilidad explotable en un sistema.
Exploit	Código o técnica que aprovecha una vulnerabilidad.
CVE	Identificador único de vulnerabilidad (MITRE).
CVSS	Puntuación estándar de severidad (0-10).
CWE	Clasificación de tipos de debilidades de software.
OWASP Top 10	Ranking de vulnerabilidades web más críticas.
Zero-day	Vulnerabilidad desconocida para la que no existe parche.
Pentesting	Prueba de penetración autorizada.
Hardening	Proceso de reducir la superficie de ataque.
Parche	Actualización que corrige una vulnerabilidad.
Falso positivo	Alerta de vulnerabilidad que no es real.
WAF	Firewall de aplicación web.

8. Autoevaluación

Respondé por escrito; luego verificá en la Sección 9.

1. Diferenciá **vulnerabilidad**, **exploit** y **riesgo** con un ejemplo original.
2. ¿Qué significan las siglas **CVE**, **CVSS** y **CWE**? ¿En qué se diferencian?
3. Elegí **dos componentes** de la infraestructura web (balanceador, WAF, servidor web, BD) y para cada uno identificá una vulnerabilidad típica y una salvaguarda.
4. En el caso de estudio de la Municipalidad, ¿cuál vulnerabilidad considerarás **la más crítica** y por qué? Justificá usando la Tríada CIA.
5. ¿Qué herramienta de las vistas en A06 usarías para detectar que el servidor Apache está desactualizado? Describí el comando.

9. Respuestas de la Autoevaluación

Nota para el estudiante

Estas respuestas son de referencia. Se valora la precisión técnica, el uso correcto de terminología y la capacidad de justificar con la Tríada CIA.

Pregunta 1: Vulnerabilidad vs. Exploit vs. Riesgo

Respuesta

- **Vulnerabilidad:** Una ventana rota en una casa. Es la debilidad.
- **Exploit:** Un ladrón que sabe cómo entrar por esa ventana rota. Es la herramienta/técnica que aprovecha la debilidad.
- **Riesgo:** La probabilidad de que un ladrón entre por esa ventana $\times$ el valor de lo que robaría. Es la combinación de probabilidad e impacto.

Ejemplo técnico:

- **Vulnerabilidad:** El servidor web usa una versión de Apache con una falla conocida de path traversal (CVE-2021-41773).
- **Exploit:** Un script Python que envía una petición HTTP especialmente construida para leer archivos fuera del directorio web.
- **Riesgo:** Alta probabilidad (el exploit es público y fácil de usar) $\times$ Alto impacto (puede leer archivos de configuración con credenciales de BD).

Pregunta 2: CVE, CVSS, CWE

Respuesta

- **CVE (Common Vulnerabilities and Exposures):** Es un **identificador único** para cada vulnerabilidad conocida. Es como el «DNI» de la vulnerabilidad. Formato: CVE-AÑO-NÚMERO. No mide severidad, solo identifica.

- **CVSS (Common Vulnerability Scoring System):** Es una **puntuación numérica** (0-10) que mide la **severidad** de una vulnerabilidad. Considera vector de ataque, complejidad, autenticación e impacto en C/I/A. No identifica, mide gravedad.
- **CWE (Common Weakness Enumeration):** Es una **clasificación por tipo** de debilidad (causa raíz). Agrupa vulnerabilidades similares por su naturaleza (ej. «todas las inyecciones SQL» son CWE-89). No identifica una vulnerabilidad específica, clasifica su categoría.

Diferencia clave: CVE identifica, CVSS mide gravedad, CWE clasifica por tipo de causa.

Pregunta 3: Vulnerabilidades en componentes de infraestructura

Respuesta

Componente 1: Base de Datos

- **Vulnerabilidad típica:** Inyección SQL (CWE-89). La aplicación no valida las entradas del usuario y permite ejecutar consultas SQL arbitrarias.
- **Salvaguarda:** Usar **consultas parametrizadas** (prepared statements) en el código de la aplicación. Implementar un WAF como capa adicional de defensa. Aplicar el principio de menor privilegio en las cuentas de BD.

Componente 2: Servidor Web

- **Vulnerabilidad típica:** Software desactualizado con CVEs conocidos sin parchar. Por ejemplo, Apache 2.4.29 con vulnerabilidades publicadas.
- **Salvaguarda:** Implementar un **programa de gestión de parches** con actualizaciones automáticas. Usar herramientas de escaneo continuo para detectar software vulnerable. Aplicar hardening del servidor (desactivar módulos innecesarios).

Pregunta 4: Vulnerabilidad más crítica del caso Municipalidad

Respuesta

La vulnerabilidad más crítica es el **archivo config.php.bak con credenciales de BD visibles**, porque:

- **Confidencialidad:** Expone credenciales que permiten acceso total a la base de datos (datos de ciudadanos).
- **Integridad:** Con las credenciales, un atacante puede modificar o eliminar registros.
- **Disponibilidad:** Puede eliminar toda la base de datos.

Aunque otras vulnerabilidades son graves (Apache desactualizado, MySQL expuesto), el archivo de backup con credenciales es un **acceso directo sin barreras**: no requiere explotar una vulnerabilidad técnica compleja, solo descargar un archivo. Es un error humano/administrativo con impacto catastrófico inmediato.

CVSS estimado: 9.8 (Crítico) — Vector de ataque: Red, Complejidad: Baja, Autenticación: Ninguna, Impacto: Alto en C/I/A.

Pregunta 5: Herramienta para detectar Apache desactualizado

Respuesta

La herramienta más directa es **Nmap** con el flag de detección de servicios y scripts de vulnerabilidades:

```
1 nmap -sV -sC --script=vuln -p 80,443 target.com
```

Explicación:

- **-sV:** Detecta la versión del servicio (en este caso, reportará «Apache 2.4.29»).
- **-sC:** Ejecuta scripts por defecto de Nmap.
- **-script=vuln:** Ejecuta todos los scripts de la categoría «vulnerabilidad», que incluyen detección de CVEs conocidos para la versión detectada.
- **-p 80,443:** Escanea solo los puertos web.

Alternativamente, **Nikto** también detecta software desactualizado:

```
1 nikto -h http://target.com
```

Nikto reportará: «El servidor web está desactualizado: Apache/2.4.29» y listará los CVEs asociados.

10. Actividad A07: Identificación de Vulnerabilidades

Consigna A07 — Identificación de Vulnerabilidades: Revisión de Caso

Metodología: Revisión de Caso.

Consigna: Realizar un informe respecto al caso dado en clase e integrar alguna herramienta de las vistas en el práctico A06.

Requisitos del informe:

1. **Identificación:** Listar cada vulnerabilidad encontrada en el caso de estudio.
2. **Clasificación:** Asignar tipo CWE, severidad CVSS estimada y componente afectado.
3. **Integración de herramienta:** Describir cómo se usaría una herramienta de A06 (Nmap, Nikto, OpenVAS, etc.) para detectar cada hallazgo. Incluir comandos o capturas.
4. **Salvaguardas:** Proponer controles técnicos y administrativos.
5. **Riesgo residual:** Evaluar el riesgo antes y después de aplicar las salvaguardas.
6. **Conclusión ética:** Reflexionar sobre la responsabilidad del profesional que identifica estas vulnerabilidades.

Subilo a la carpeta del Trabajo Integrador en Google Drive, otorgando permisos de comentario a era.profe@gmail.com.

Rúbrica de evaluación (según grilla de la cátedra):

Criterio	Peso	Qué se espera para el «Excelente» (4)
A. Recopilación de información	30 %	Utiliza de manera completa fuentes confiables (documentación CVE, OWASP, man pages de herramientas) y las cita correctamente.
B. Elaboración de documentos	30 %	Cumple adecuadamente con los formatos atento a la planilla de estandarización brindada por la cátedra.
C. Elaboración de presentaciones	10 %	Argumenta de manera sólida el inconveniente del caso y su solución.
F. Consciencia de buenas prácticas	30 %	Evidencia permanentemente el uso de buenas prácticas en el análisis y la documentación.

11. Fuentes y bibliografía

11.1. Bibliografía obligatoria (según programa)

- Chicano Tejada, E. (2023). *Auditoría de seguridad informática. IFCT0109* (2.^a ed.). IC Editorial — eLibro.
- Elgueta, R. (s.f.). *Guías de Clase unidad IV*. Material inédito. UCH.

11.2. Bibliografía complementaria (según programa)

- Bancal, D. (2022). *Seguridad informática* (5.^a ed.). Ediciones ENI.
- Evans, L. (2019). *Ethical Hacking: The Ultimate Guide to Using Penetration Testing to Audit and Improve the Cybersecurity of Computer Networks for Beginners, Including Tips on Social Engineering*. Amazon Digital Services LLC — KDP.
- White, A. & Clark, B. (2017). *BTFM: Blue Team Field Manual*. CreateSpace Independent Publishing Platform.
- ISO/IEC (2005). *Information security management 27001*.

11.3. Bibliografía específica de vulnerabilidades

- OWASP Foundation. (2021). *OWASP Top 10:2021*.
- MITRE Corporation. *CVE Dictionary*: <https://cve.mitre.org/>
- NIST. *National Vulnerability Database (NVD)*: <https://nvd.nist.gov/>
- FIRST.org. *CVSS v3.1 Specification Document*.

11.4. Recursos web y laboratorios

- TryHackMe (laboratorio de la materia): <https://tryhackme.com>
- Sala recomendada para esta clase: <https://tryhackme.com/room/vulniversity>
- Nmap Reference Guide: <https://nmap.org/book/man.html>
- Nikto Documentation: <https://cirt.net/nikto2>
- Searchsploit / Exploit-DB: <https://www.exploit-db.com/>